Problemas Clássicos de IPC

Professor: Paulo Roberto de Oliveira
Disciplina: Sistemas Operacionais

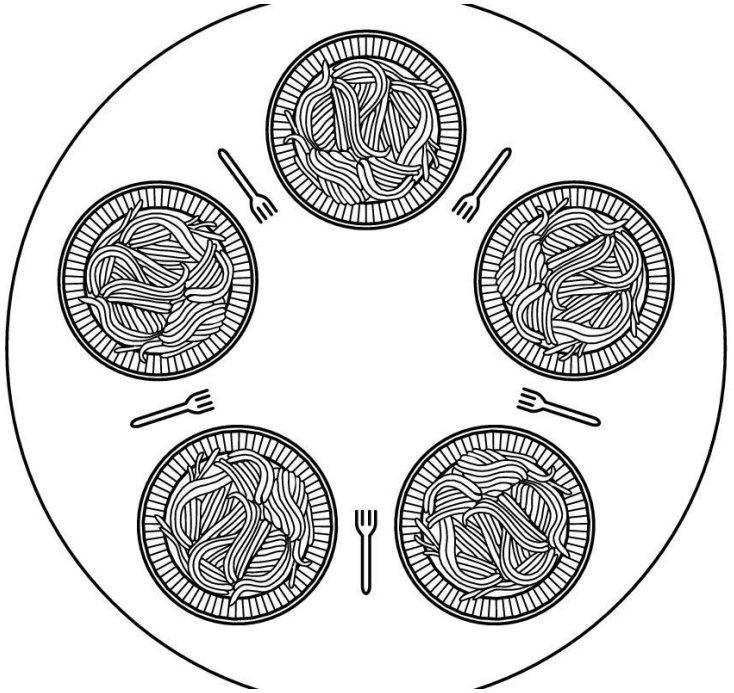
Departamento de Informática
Universidade Estadual de Maringá

O Jantar dos Filósofos

- Em 1965, Dijkstra propôs e resolveu um problema de sincronização que ele chamou de Jantar dos Filósofos.
- O sistema é bastante simples: 5 filósofos estão sentados ao redor de uma mesa, cada um com um prato de macarrão a sua frente. Entre cada 2 filósofos existe um garfo.
- Os filósofos podem estar em um de dois estados possíveis: pensando ou comendo. Para comer o macarrão, o filósofo precisa de dois garfos.

O Jantar dos Filósofos

- Quando um filósofo fica com fome, ele tenta obter dois garfos. Se conseguir, ele pode comer. Se um ou os dois garfos não estiverem disponíveis (estão com algum outro filósofo), é preciso esperar.



O Jantar dos Filósofos

- Uma solução comum é esta:

```
#define N 5                          /* number of philosophers */

void philosopher(int i)              /* i: philosopher number, from 0 to 4 */
{
    while (TRUE) {
        think( );                    /* philosopher is thinking */
        take_fork(i);                /* take left fork */
        take_fork((i+1) % N);        /* take right fork; % is modulo operator */
        eat( );                      /* yum-yum, spaghetti */
        put_fork(i);                 /* put left fork back on the table */
        put_fork((i+1) % N);         /* put right fork back on the table */
    }
}
```

O Jantar dos Filósofos

- Infelizmente, esta solução não funciona.
- Suponha que todos os 5 tentem pegar o garfo esquerdo simultaneamente. Nenhum deles poderá pegar o garfo direito e temos um deadlock.

O Jantar dos Filósofos

- Uma tentativa de melhorar esta solução é colocar um teste no segundo *take_fork*. Se ele não estiver disponível, o primeiro garfo é solto.
- Isso também não funciona.
- Suponha que todos peguem o garfo da esquerda. Testando o da direita, todos verificam que ele não está presente e soltam o primeiro em seguida, todos tentam novamente resultando num impasse.
- Esta situação é chamada de **starvation**.

O Jantar dos Filósofos

- Uma outra possibilidade é que os processos esperem um período de tempo aleatório antes de tentar de novo. A chance de dois processos escolherem o mesmo período é bem menor. Há aplicações que funcionam dessa forma.
- Porém, algumas aplicações podem precisar de uma solução que funcione sempre (p. ex.: sistemas críticos).

O Jantar dos Filósofos

- Uma solução que não possui deadlocks nem starvation é proteger os 5 comandos que seguem a chamada think() com um semáforo binário. Antes de tentar pegar o primeiro garfo, o processo chama um down sobre o mutex. Depois de soltar os garfos, o processo chama um up.
- Esta solução tem um problema prático: apenas um processo pode comer por vez. Com 5 processos (filósofos), pelo menos 2 poderiam comer por vez.
- A solução efetiva para este problema pode ser encontrada no livro e não é tão simples..

Leitores e Escritores

- Este problema modela o acesso a bancos de dados.
- Com múltiplos processos acessando um BD, é aceitável que vários deles possam acessar simultaneamente desde que sejam apenas de leitura.
- Porém, quando um processo acessa o BD para atualização, nenhum outro pode acessar simultaneamente, nem mesmo os de leitura.

Leitores e Escritores

- Esse problema apresenta duas possibilidades:
- **1)** Suponha que alguns leitores estão acessando o BD e chega um processo de atualização:
  - Este último é bloqueado – enquanto houver processo de leitura, e assim o que atualiza ficará esperando indefinidamente;
  - Caso cheguem mais processos de leitura, o que atualiza continuará esperando (virtualmente para sempre);
  - Essa situação indica que os processos de leitura têm prioridade sobre os de atualização.

Leitores e Escritores

- **2)** Pode-se dar prioridade para os processos de atualização:
  - Supondo que o processo de atualização bloqueie a entrada de novos processos de leitura;
  - Nesse caso, os processos de leitura atuais eventualmente terminarão, liberando o acesso ao processo de atualização.
- Este problema trata de diferentes tipos de tratamentos para diferentes classes de processos.

Leitores e Escritores

```
typedef int semaphore;          /* use your imagination */
semaphore mutex = 1;            /* controls access to 'rc' */
semaphore db = 1;               /* controls access to the database */
int rc = 0;                     /* # of processes reading or wanting to */

void reader(void)
{
    while (TRUE) {              /* repeat forever */
        down(&mutex);           /* get exclusive access to 'rc' */
        rc = rc + 1;            /* one reader more now */
        if (rc == 1) down(&db); /* if this is the first reader ... */
        up(&mutex);             /* release exclusive access to 'rc' */
        read_data_base( );      /* access the data */
        down(&mutex);           /* get exclusive access to 'rc' */
        rc = rc – 1;            /* one reader fewer now */
        if (rc == 0) up(&db);   /* if this is the last reader ... */
        up(&mutex);             /* release exclusive access to 'rc' */
        use_data_read( );       /* noncritical region */
    }
}
```

```
void writer(void)
{
    while (TRUE) {              /* repeat forever */
        think_up_data( );       /* noncritical region */
        down(&db);              /* get exclusive access */
        write_data_base( );     /* update the data */
        up(&db);                /* release exclusive access */
    }
}
```

O Barbeiro Dorminhoco

- Este problema apresenta a seguinte situação:
  - Há uma barbearia com um barbeiro, uma cadeira de barbeiro e N cadeiras de espera;
  - Se não há clientes, o barbeiro senta na cadeira de barbeiro e dorme;
  - Quando chega o primeiro cliente, ele acorda o barbeiro, ocupa a cadeira e tem o seu cabelo cortado;
  - Caso cheguem outros clientes, eles devem ocupar as cadeiras de espera;
  - Por fim, se chega um novo cliente e há alguém sendo atentido pelo barbeiro e todas as cadeiras de espera estão ocupadas, ele deve deixar o sistema;
  - Quando o barbeiro termina de atender o cliente atual, este sai da cadeira e vai embora;
  - Nesse caso, um dos clientes da espera é escolhido;
  - Caso não haja mais nenhum, o barbeiro volta a dormir.

O Barbeiro Dorminhoco

O Barbeiro Dorminhoco

- Este modelo é semelhante a vários problemas onde se apresenta uma fila para acessar algum tipo de serviço.
- Se trata de uma prestação de serviços a clientes (cliente-servidor).
- Soluções para melhorar o atendimento:
  - Aumentar a velocidade do barbeiro;
  - Aumentar o número de barbeiros;
  - Reduzir o trabalho feito pelo barbeiro para cada cliente (algoritmo mais eficiente).

O Barbeiro Dorminhoco

```
#define CHAIRS 5                    /* # chairs for waiting customers */

typedef int semaphore;              /* use your imagination */

semaphore customers = 0;            /* # of customers waiting for service */
semaphore barbers = 0;              /* # of barbers waiting for customers */
semaphore mutex = 1;                /* for mutual exclusion */
int waiting = 0;                    /* customers are waiting (not being cut) */

void barber(void)
{
    while (TRUE) {
        down(&customers);           /* go to sleep if # of customers is 0 */
        down(&mutex);               /* acquire access to 'waiting' */
        waiting = waiting – 1;      /* decrement count of waiting customers */
        up(&barbers);               /* one barber is now ready to cut hair */
        up(&mutex);                 /* release 'waiting' */
        cut_hair( );                /* cut hair (outside critical region) */
    }
}
```

```
void customer(void)
{
    down(&mutex);                   /* enter critical region */
    if (waiting < CHAIRS) {         /* if there are no free chairs, leave */
        waiting = waiting + 1;      /* increment count of waiting customers */
        up(&customers);             /* wake up barber if necessary */
        up(&mutex);                 /* release access to 'waiting' */
        down(&barbers);             /* go to sleep if # of free barbers is 0 */
        get_haircut( );             /* be seated and be serviced */
    } else {
        up(&mutex);                 /* shop is full; do not wait */
    }
}
```

Dúvidas?